Guess Market - Exercise 4 (bonus): the web client

Name: Dan Alushe
ID: 212472864
Email: danalushe15@gmail.com
GitHub: https://github.com/danal15/guess-market

The URL to open

http://localhost:3000
Open this address after starting the server and the web client.

How to run

- Put guess-market.war in tomcat\webapps and start Tomcat. This is the same exercise 3 server and it is not changed for this exercise.
- Double click run-web.bat. It starts the small Node server for the web client. Leave the window open while using it.
- Open http://localhost:3000 in Chrome, Edge or Firefox.
- Log in with any name. There are no passwords, and a name that is already in use is refused.
- For two browser users, use an incognito/private window or another browser. Normal tabs share the same session cookie. The exercise 3 JavaFX client can also run at the same time.
- Node 16 or newer and a current browser are required for the web client. The JavaFX client (desktop-client\run.bat) needs Java 25. There are no npm packages and nothing else has to be installed.
- If Tomcat is not on localhost:8080, pass its address to run-web.bat. If port 3000 is taken, set PORT to another port before running it.

How to use it

- Events tab - view only. Shows all events with the same method / status / commission filters. Clicking an event shows its details, trades, order books when relevant, and charts.
- Account tab - shows your balance, money movements, balance chart, the other users and the events you can act on.
- The available actions are the same as the JavaFX client: create an event, open it, buy LMSR shares, place an order, and close an event. The web client does not upload event files.
- The trade forms ask the server for the current cost while you type. A disabled action stays visible and has a tooltip explaining why.
- Everything refreshes once a second. If the server goes away the page shows it in the header and keeps trying.
- The exercise 3 chat is also available in the drawer at the bottom.

Assumptions and decisions

- The Node server serves the page and proxies /guess-market requests to Tomcat. This avoids the browser same-origin and cookie problem without changing the exercise 3 war.
- Reloading the page does not log the user out. The page checks the existing server session first.
- Nothing is saved. Restarting Tomcat clears the users, events and chat, the same as exercise 3.
- The web client is English only.

Project structure

run-web.bat and web\ - the new web client.
guess-market.war - the exercise 3 server, unchanged.
desktop-client\ - the exercise 3 JavaFX client, included so both clients can be tested against the same server.
web\server.js - serves the web files and proxies requests to Tomcat.

web\public\index.html - the login, events and account screens.
web\public\app.js - the client logic and calls to the server.
web\public\style.css - the page styling.
The client is plain HTML, CSS and JavaScript with no framework or libraries. The charts are SVG. Text that comes from users or files is added as text and not as HTML.

Working with the AI tool

- I used Claude through Claude Code. I also tried GitHub Copilot, but Claude Code was more useful here because it could read the whole project, edit several files and run the application.
- The first version was not correct even though the code looked reasonable. I had to test it in a real browser. One problem was the login screen staying visible because of CSS, and another was the browser same-origin problem that led to the small Node proxy.
- In exercise 3 my client could not log in. Java's HTTP client uses HTTP/2 by default, and Tomcat rejected the upgrade request with a 400 error. I could not find the cause myself; Claude Code found it by reproducing the failure and testing HTTP/1.1 against HTTP/2, and the fix was one line.
- The AI did most of the typing, but I made the decisions and checked whether the result actually worked. I would estimate about three quarters was written from prompts and the rest was testing, fixing and deciding what should happen.
- It took about a day and a half. Most of the time was testing and fixing, not writing the first version.
- I had almost no front end experience before this. I had used a little HTML before, but not real JavaScript, so I checked things by running them instead of trusting the code just from reading it.
- I enjoyed this exercise more than I expected. It was different from writing the earlier exercises myself because it felt more like reviewing and fixing somebody else's work. The useful part was how fast it could produce a first version; the annoying part was that wrong answers often looked just as convincing as correct ones.

What stayed the same

The market rules and the exercise 3 server behavior are unchanged. The web client uses the same server endpoints as the JavaFX client.